

Software Language Engineering (SLE): Exercises

Erwan Bousse

Version 2026-2027, 2026-10-22

Table of Contents

1. ✎ Exercises: designing an abstract syntax . . . 1
   1.1. Exercice 1: component architecture DSL . . . 1
   1.2. Exercice 2: robot programming DSL . . . 1
2. ✎ Exercises: designing a concrete syntax . . . 5
   2.1. Exercice 1: component architecture DSL . . . 5
   2.2. Exercice 2: robot programming DSL . . . 5
3. ✎ Exercises: designing a semantics . . . 7

Chapter 1. Exercises: designing an abstract syntax

1.1. Exercice 1: component architecture DSL

We would like to design a small *component architecture DSL* that can be used to design the different types of components of a software architecture. The goal is to be able to define software components, and their relationships, without (yet) considering what programming langauges and framework will be used for their implementation.

The expected DSL should allow the definition of an *architecture*, which contains a set of connector interfaces, a set of component types, and a system (where instances of component types are created and linked).

- A *connector interface* simply has a name.



A more realistic DSL should also allow the definition of possible operations or messages allowed by a given connector interface, to design how actual data or control is sent between components.

- A *component type* has a name, and contains a set of connectors.
  - A *connector* has a name, a connector interface, and can either *provide* or *require*.
- A *system* has a set of component instances, and a set of links.
  - A *component instance* has a name, references a component type.
  - A *link* has a source component instance, a provides connector (of said source instance), a target component instance, and a requires connector (of said target instance).

1.1.1. Question 1

Propose an **abstract syntax** design for this DSL using a MOF metamodel.

1.1.2. Question 2

Consider that we have a *web application* architecture, where a frontend uses both a domain API provided by a backend, and a DOM API provided by a web browser. In addition, the backend uses an *alert* service provided by the frontend to trigger the spontaneous display of an alert message.

Using your abstract syntax, give an **abstract syntax tree (AST)** corresponding to this description.

1.1.3. Question 3

The current abstract syntax can be used to create *wrong* architecture models. Why?

What addditional *constraints* should be formalized with our abstract syntax to ensure no wrong models could be made?

1.2. Exercice 2: robot programming DSL

1.2.1. Version 1

We would like to design a small *robot programming DSL* that can be used to program small moving robots with motors and lights. A robot program should define a sequence of actions (moving, rotating, turning light on, etc.) that the robot shall perform when asked.

The DSL should first allow the definition of a *robot simulation*, which has a a name, a set of declared robots, and a set of declared programs.

- A *robot* is declared with a name, and with a set of *components*. A component also has a name, and can be either a *light* or a *motor*. A *motor* has a max rotation speed (*revolutions per minute* or *rpm*).
- A *program* is declared with a name, and is specific to a robot already defined. A program is a sequence of *commands* which can be of different types:
  - *changing the color* of a given light of the robot (possibilities are: green, red, blue, yellow, none),
  - *moving* a given distance in centimeters (forward or backward),
  - *rotating* a given angle in degrees (left or right).

Question 1

Propose an abstract syntax design for this DSL using a MOF metamodel.

Question 2

Consider that we have a simulation with one robot R1_, two lights (*l1* and *l2*), two motors (*m1* and *m2*) with both a max rotation speed of 2000. A program *p1* is defined for this robot where the robot moves forward 10 centimeters, rotates 45° right, sets its first light to yellow, and moves forward 20 centimeters.

Using your abstract syntax, give an *abstract syntax tree* (AST) corresponding to this description.

1.2.2. Version 2

A problem with our DSL is that *move forward* and *rotate* are actions that are very robot dependent:

- **regarding *forward movement***, if the robot has two parallel wheels, then a rotation means rolling each wheel in the same rotation direction (eg. *clockwise*) ; but if the robot has three wheels in a triangle fashion, than only two wheels should be in movement in the same rotation direction.
- **regarding *rotation***, if the robot has two parallel wheels, then a rotation means rolling each wheel in a different rotation direction (one *clockwise* and one *counterclockwise*) ; but if the robot has three wheels in a triangle fashion, then all wheels should roll in the same rotation direction.

Moreover, moving 1 cm forward, or rotating 1°, will be different for each robot based on the size of wheels and the motor characteristics: the *duration* and *power* used to trigger each motor will be different in each case.

Therefore we must find a way for our DSL to allow specifying, for each robot, what moving forward *means*, and what rotating *means*, both in terms of motor manipulation.

Question 3

Modify the V1 abstract syntax to integrate new concepts that allow defining, for each robot, how forward movement and rotation are mapped to the declared motors of the robot.

Question 4

Modify the V1 example AST to add new objects that define forward movement and rotation for the robot R1:

- for R1, moving 1cm forward means activating both motors during 0.2 seconds with a power of 100W,
- for R1, rotating 1° means activating both motors during 0.2 seconds with a power of 50W, with one motor rotating in *reverse*.

Chapter 2. Exercises: designing a concrete syntax

2.1. Exercice 1: component architecture DSL

We continue the design of the component architecture DSL, started in Chapter 1. In the following questions we assume we already have an **abstract syntax** design for this DSL.

2.1.1. Question 1

After dialoguing with (imaginary) domain experts that will use this DSL, we learn they have the following preferences regarding a textual concrete syntax of the DSL:

- they prefer lowercase keywords,
- they prefer keywords over operators (),
- they prefer whole words (*requires*, *provides*) over abreviations (*req*, *prov*),
- they are familiar with braces {} as delimiters,
- they have experience with programming languages such as C, C++, Java.

Propose a **mockup** concrete syntax representation of the AST you made in Chapter 1 for this DSL.

2.1.2. Question 2

In the mockup proposed in the previous question, give the **token type** (*keywords*, *operators*, *delimiters*, *separators*, *identifiers*, *comments*, *literals*) of each appearing token.

2.1.3. Question 3

Using the proposed mockup, create one **annotated mockup** for each concept of the abstract syntax, where you highlight the textual representation of each property/association of the concept.

2.2. Exercice 2: robot programming DSL

We continue the design of the robot programming DSL, started in Chapter 1. In the following questions we assume we already have an **abstract syntax** design for this DSL.

2.2.1. Question 1

After dialoguing with (imaginary) domain experts that will use this DSL, we learn they have the following preferences regarding a textual concrete syntax of the DSL:

- they like concise notations (for example using operators),
- they prefer abbreviations over whole words, when relevant,
- they are not familiar with programming languages.

Propose a **mockup** concrete syntax representation of the AST you made in Chapter 1 for this DSL.

2.2.2. Question 2

In the mockup proposed in the previous question, give the **token type** (*keywords*, *operators*, *delimiters*,

separators, *identifiers*, *comments*, *literals*) of each appearing token.

2.2.3. Question 3

Using the proposed mockup, create one **annotated mockup** for each concept of the abstract syntax, where you highlight the textual representation of each property/association of the concept.

Chapter 3. ✏ Exercises: designing a semantics

TODO